

Homework 2-2:

Intro. to Virtual and Augmented Reality

Simple Character Modeling & Animation

학번: 2025021826

이름: 정경서

목차

1. 과제 개요
2. 캐릭터 구조 및 Scene 계층
3. 관절 좌표계 배정
4. 변환 행렬 T 계산
5. 관절 회전 효과: 최종 T
6. Unity import 및 Animation 구현

1. 과제 개요

본 보고서는 HW2 중 과제 2번 (Simple Character Modeling)을 다룬다. Blender를 사용하여 단순한 사람형 캐릭터를 도형 조합으로 모델링하고, 관절(Joint) 위치에 Empty 오브젝트를 배치하여 각 관절의 로컬 좌표계를 설정하였다. 좌표계 배정 및 변환 행렬 계산은 Correction 공지에 명시된 단순화된 방법을 따랐다(DH 파라미터 방법은 사용하지 않음). 이후 FBX 포맷으로 Unity에 import하여 C# Script로 Body Animation (걷기, 인사, 손 흔들기) 및 Facial Expression Animation (기쁨, 슬픔, 화남 및 찡그림, 말하기, 눈 깜빡임)을 구현했다.

과제 관련 Unity 구현은 [GitHub \(https://github.com/KyungseoJung/Unity_Personal_VR2026_HW1-2_360Scene_JKS\)](https://github.com/KyungseoJung/Unity_Personal_VR2026_HW1-2_360Scene_JKS)의 Scene_HW2-2이름의 Scene에서 확인 가능하다.

추가로, exe 파일은 함께 첨부한 VR2026_HW2-2_JKS_Build(Window).zip에서 확인 가능하다.

항목	수행 내용
캐릭터 모델링	머리, 몸통, 양팔, 양다리, 눈, 눈썹, 입으로 구성된 단순 인간형 캐릭터 제작
Joint Hierarchy	J1_Root, J2_Neck, J3_RightShoulder, J4_LeftShoulder, J5_RightHip, J6_LeftHip 구성
좌표계 배정	각 joint의 local x/y/z축을 정의. z축은 회전축, x축은 link 방향, y축은 자동 결정
T matrix 작성	World와 J1, 그리고 J1과 J2~J6 사이의 4x4 transformation matrix 작성
Unity Import	Blender에서 제작한 모델을 Unity에 import하고 joint hierarchy가 유지되는지 확인
Animation script	Body Animation 3가지(걷기, 인사, 손 흔들기)와 Facial Expression Animation 5가지(기쁨, 슬픔, 화남 및 찡그림, 말하기, 눈 깜빡임) 구현

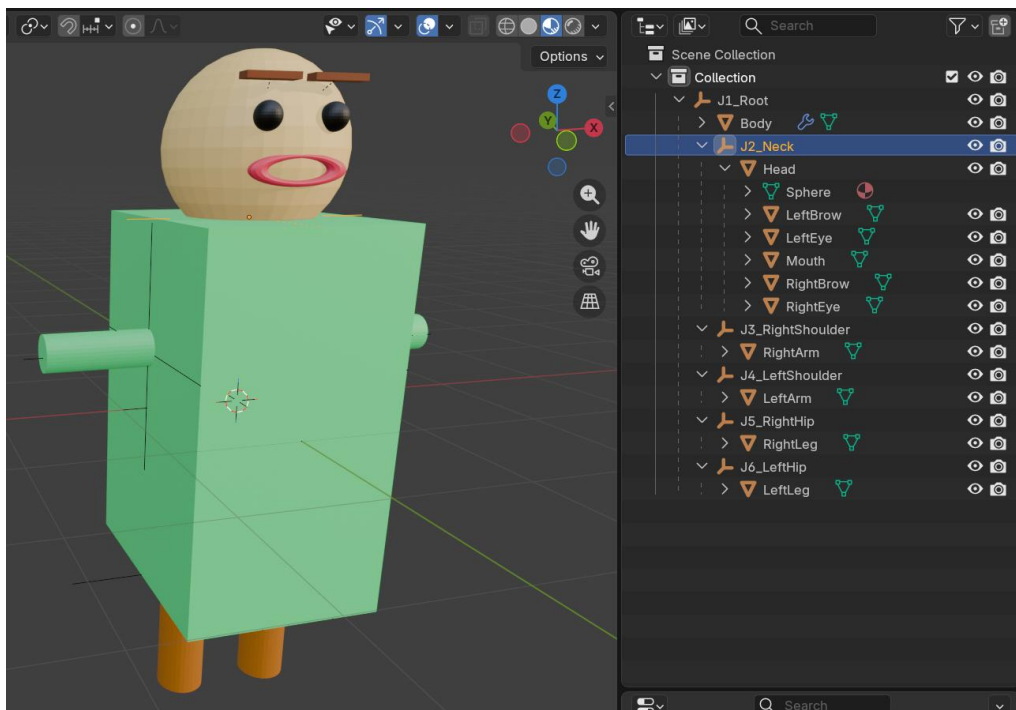
2. 캐릭터 구조 및 Scene 계층

a. 설계 방식 - Empty 오브젝트를 Joint로 활용

각 관절의 회전 중심 위치에 Blender의 Empty(Plain Axes) 오브젝트를 생성하고, Mesh 오브젝트를 해당 Empty의 자식으로 연결했다. Mesh 오브젝트의 원점과 실제 관절 위치가 일치하지 않는 문제를 해결 및 방지하기 위한 방법으로, Unity에서 Joint Empty를 회전시키면서 Mesh가 올바르게 따라 움직이도록 구조를 만들었다.

b. Blender Scene 계층 구조 (Outliner)

J1_Root	(Empty — 골반/전신 기준점)
├── Body	(Mesh — 몸통, Cube)
├── J2_Neck	(Empty — 목/머리 회전 기준)
│ ├── Head	(Mesh — 머리, Sphere)
│ ├── LeftBrow	(Mesh — 왼쪽 눈썹, Cube)
│ ├── LeftEye	(Mesh — 왼쪽 눈, Sphere)
│ ├── Mouth	(Mesh — 입, Torus)
│ ├── RightBrow	(Mesh — 오른쪽 눈썹, Cube)
│ └── RightEye	(Mesh — 오른쪽 눈, Sphere)
├── J3_RightShoulder	(Empty — 오른쪽 어깨)
│ └── RightArm	(Mesh — 오른팔, Cylinder)
├── J4_LeftShoulder	(Empty — 왼쪽 어깨)
│ └── LeftArm	(Mesh — 왼팔, Cylinder)
├── J5_RightHip	(Empty — 오른쪽 엉덩이)
│ └── RightLeg	(Mesh — 오른다리, Cylinder)
└── J6_LeftHip	(Empty — 왼쪽 엉덩이)
└── LeftLeg	(Mesh — 왼다리, Cylinder)



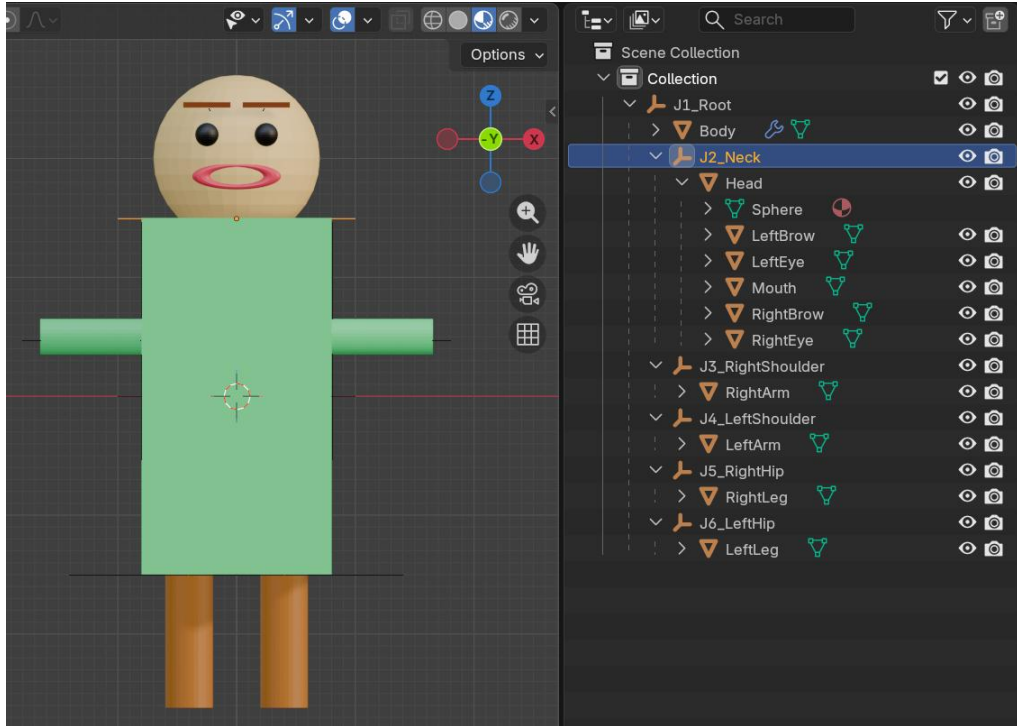
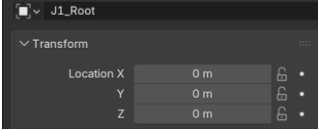
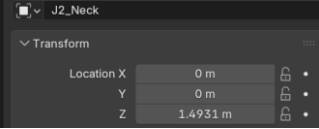
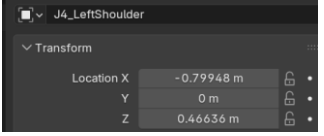
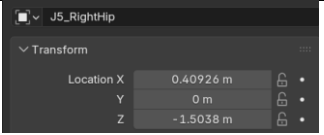
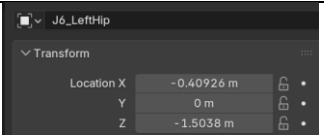


Figure 1: 완성된 Blender 캐릭터 전면 화면

c. 각 Joint의 위치값 (Blender Properties에서 측정)

아래 표는 Blender Transform 패널에서 확인한 각 Joint Empty의 Location 값을 작성한 것이다. 이 값은 각 joint의 origin 위치이다. 본 보고서에서는 Blender 좌표계 기준으로 +X는 캐릭터 오른쪽, -X는 왼쪽, +Z는 위쪽, -Z는 아래쪽, -Y는 얼굴이 향하는 앞쪽, +Y는 뒤쪽으로 정의하였다.

Joint 이름	X	Y	Z	설명	Transformation 스크린샷
J1_Root	0	0	0	몸통/ root 기준점	
J2_Neck	0	0	1.4931	목 관절 위치	
J3_RightShoulder	0.809	0	0.466	오른쪽 어깨 관절 위치	

J4_LeftShoulder	-0.799	0	0.466	왼쪽 어깨 관절 위치	
J5_RightHip	0.409	0	-1.504	오른쪽 엉덩이 관절 위치	
J6_LeftHip	-0.409	0	-1.504	왼쪽 엉덩이 관절 위치	

3. 관절 좌표계 배정

위에서 작성한 joint 위치와 같은 방식으로, 변환행렬도 Blender 좌표계를 기준으로 작성하였다. Blender에서 캐릭터의 오른쪽은 +X, 왼쪽은 -X, 위쪽은 +Z, 아래쪽은 -Z, 얼굴이 향하는 앞쪽은 -Y, 뒤쪽은 +Y로 정의하였다. 그리고, J1_Root는 World coordinate system과 일치하도록 두었다. 따라서 $W_T_1 (^W T_1)$ 은 항등행렬이다.

a. Link 방향과 회전축 방향 이해, Correction 공지 방법 적용

Link 방향은 joint에서 연결된 body part가 뻗어나가는 방향이다. 예를 들어, J3_RightShoulder에서 RightArm은 캐릭터 오른쪽으로 뻗어 있으므로 link 방향은 +X이다. J5_RightHip에서 RightLeg는 아래쪽으로 내려가므로 link 방향은 -Z이다. 회전축 방향은 해당 joint가 회전할 때 기준이 되는 중심선의 방향이다. Link 방향과 회전축 방향은 일반적으로 다르다. 예를 들어, 오른팔은 +X 방향으로 뻗어 있지만, 팔을 위아래로 들어 올리는 동작은 앞뒤 방향의 축을 기준으로 회전하기 때문에, 오른팔의 link 방향은 +X이고, 회전축은 -Y로 설정할 수 있다.

추가로 공지된 Correction 공지에 따라, 본 보고서에서는 DH 파라미터 방법을 사용하지 않고, 아래의 단순화된 방법으로 각 관절의 local z축을 회전축 방향으로 정의하였다. 가능한 경우 local x축은 link방향으로 정의하였고, local y축은 $(x \times y = z)$ 를 만족하도록 오른손 좌표계에 의해 결정하였다.

적용한 좌표계 배정 규칙은 아래와 같다.

규칙 1: z축 = 해당 관절이 회전하는 축 (회전축)

규칙 2: x축 = 다음 링크(뼈대)가 뻗어나가는 방향

규칙 3: y축 = 오른손 좌표계 법칙으로 자동 결정

b. 각 관절의 축 방향 배정표

다음 표는 본 모델에서 사용한 joint local coordinate system이다. 본 보고서에서는 Blender 좌표계를 기준으로 +X를 캐릭터 오른쪽, -X를 왼쪽, +Z를 위쪽, -Z를 아래쪽, -Y를 얼굴이 향하는 앞쪽, +Y를 뒤쪽으로 정의했다. 여기서 local z축은 각 관절의 회전축으로 배정하였다. local x축은 가능한 경우 link 방향, 즉 해당 관절에서 연결된 body part가 뻗어나가는 방향으로 배정했다. local y축은 오른손 좌표계 법칙 $x \times y = z$ 를 만족하도록 결정했다.

여기서 주의할 점은, local z축이 Blender의 global +Z 방향을 의미하는 것이 아니라는 점이다. 예를 들어, J3_RightShoulder의 local z축을 -Y로 둔다는 말은, J3의 (local) z축이라는 이름의 축이 Blender의 global -Y축 방향을 향하도록 수학적으로 정의했다는 뜻이다.

Joint	위치	local x축	local y축	local z축	설명
J1_Root	골반/몸통 중심	+X	+Y	+Z	root 기준 좌표계
J2_Neck	목, 머리 연결	-Y	+X	+Z	고개 좌우 회전축: 위쪽(+Z)
J3_RightShoulder	오른쪽 어깨	+X	+Z	-Y	오른팔 link 방향은 +X, 팔을 위아래로 움직이는 회전축은 앞쪽 (-Y)
J4_LeftShoulder	왼쪽 어깨	-X	-Z	-Y	왼팔 link 방향은 -X, 팔을 위아래로 움직이는 회전축은 앞쪽 (-Y)
J5_RightHip	오른쪽 엉덩이	-Z	+Y	+X	오른다리 link 방향은 아래쪽(-Z), 앞뒤로 걷기 swing 회전축은 좌우 방향(+X)
J6_LeftHip	왼쪽 엉덩이	-Z	+Y	+X	왼다리 link 방향은 아래쪽(-Z), 앞뒤로 걷기 swing 회전축은 좌우 방향(+X)

4. 변환 행렬 T 계산

T는 transformation matrix, 즉 변환행렬을 의미하며, 특정 child joint 좌표계에서 표현된 점을 parent joint 좌표계 기준으로 변환할 때 사용한다. 예를 들어 (1T_3)는 J3_RightShoulder 좌표계의 점을 J1_Root 좌표계로 변환하는 4x4 행렬이다.

$$p_1 = ({}^1T_3) p_3$$

여기서 p_3 는 J3 좌표계에서 표현된 점이고, p_1 은 같은 점을 J1 좌표계에서 표현한 결과이다.

변환행렬 T는 회전 성분 R과 이동 성분 t로 구성된다.

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix}$$

$$\mathbf{T} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$\mathbf{R}$ 은 child joint의 local x,y,z축이 parent 좌표계에서 어느 방향을 향하는지 나타내는 3x3 rotation matrix이고, $\mathbf{t}$ 는 parent joint의 origin에서 child joint의 origin까지의 이동을 나타내는 translation vector이다. 본 모델에서는 J1_Root와 World를 일치시켜 (${}^W\mathbf{T}_1$)을 항등 행렬(identity matrix)로 두었다. 그리고 J2~J6은 모두 J1_Root의 child이므로, 각 $\mathbf{t}$ 는 Blender Transform 패널에서 측정한 J2~J6의 Location 값을 그대로 사용한다.

구체적인 변환 행렬 계산 방법은 아래와 같다.

- 1) **T1 계산** (초기 자세 - 관절이 아무것도 안 돌아간 상태)

(‘자식 joint frame이 부모 joint frame 기준으로 어디에 있고, 어떤 방향을 보고 있는가?’를 나타냄)

$\mathbf{R}$ 구성: 자식 좌표계의 local x, y, z축을 부모 좌표계 기준으로 표현하여 열에 배치 → 3x3 행렬

$\mathbf{t}$ 구성: 부모의 origin → 자식의 origin까지의 이동 벡터 (Blender 측정값 사용)

$$\mathbf{T}_1 = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix}$$

- 2) **T2 계산** (관절이 θ 만큼 z축 회전)

(joint 회전 효과를 나타냄. 과제 공지에 따라, 관절 회전은 local z축 회전으로 설정)

$$\mathbf{T}_2 = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & | & 0 \\ \sin\theta & \cos\theta & 0 & | & 0 \\ 0 & 0 & 1 & | & 0 \\ 0 & 0 & 0 & | & 1 \end{bmatrix}$$

- 3) **최종 $\mathbf{T} = \mathbf{T}_2 \times \mathbf{T}_1$**

(최종 변환 = joint 회전 효과 x 초기 자세 변환)

a. World → J1_Root (${}^W\mathbf{T}_1$)

과제 조건(좌표계 1(J1_Root)과 W(World)가 일치하도록 설정)에 따라, (${}^W\mathbf{T}_1$)은 항등 행렬이다.

1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

참고로 Unity import 시, Blender와 Unity 간의 좌표계 차이 (Blender: Z-up, Unity: Y-up)로 인해 캐릭터 root에 X축 -90도 회전 보정을 적용하였다. 이에 대한 자세한 내용은 [6. Unity Import 및 Animation 구현]에서 설명한다.

b. J1_Root → J2_Neck (1T_2)

J2_Neck의 축은 $x_2 = -Y$, $y_2 = +X$, $z_2 = +Z$ 로 정의했다. 이를 J1 기준 벡터로 쓰면, $x_2 = (0, -1, 0)$, $y_2 = (1, 0, 0)$, $z_2 = (0, 0, 1)$ 이다.
그리고 J2_Neck의 위치는 $t = (0, 0, 1.4931)$ 이다.

$$R = [x_2, y_2, z_2]$$

$$= \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$t = [0, 0, 1.4931]^T$$

T_1 (초기 자세)은 아래와 같다.

0	1	0	0
-1	0	0	0
0	0	1	1.4931
0	0	0	1

c. J1_Root → J3_RightShoulder (1T_3)

J3_RightShoulder의 축은 $x_3 = +X$, $y_3 = +Z$, $z_3 = -Y$ 로 정의했다. 이를 J1 기준 벡터로 쓰면, $x_3 = (1, 0, 0)$, $y_3 = (0, 0, 1)$, $z_3 = (0, -1, 0)$ 이다.
그리고 J3_RightShoulder의 위치는 $t = (0.809, 0, 0.46636)$ 이다.

$$R = [x_3, y_3, z_3]$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & -1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$$

$$t = [0.809, 0, 0.46636]^T$$

T1(초기 자세)은 아래와 같다.

1	0	0	0.809
0	0	-1	0
0	1	0	0.46636
0	0	0	1

d. J1_Root → J4_LeftShoulder (1T_4) (1T_4)

J4_LeftShoulder의 축은 $x_4 = -X$, $y_4 = -Z$, $z_4 = -Y$ 로 정의했다. 이를 J1 기준 벡터로 쓰면, $x_4 = (-1, 0, 0)$, $y_4 = (0, 0, -1)$, $z_4 = (0, -1, 0)$ 이다.

그리고 J4_LeftShoulder의 위치는 $t = (-0.79948, 0, 0.46636)$ 이다.

$$R = [x_4, y_4, z_4]$$

$$= \begin{bmatrix} -1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & -1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & -1 & 0 \end{bmatrix}$$

$$t = [-0.79948, 0, 0.46636]^T$$

T1(초기 자세)은 아래와 같다.

-1	0	0	-0.79948
0	0	-1	0
0	-1	0	0.46636
0	0	0	1

e. J1_Root → J5_RightHip (1T_5) (1T_5)

J5_RightHip의 축은 $x_5 = -Z$, $y_5 = +Y$, $z_5 = +X$ 로 정의했다. 이를 J1 기준 벡터로 쓰면, $x_5 = (0, 0, -1)$, $y_5 = (0, 1, 0)$, $z_5 = (1, 0, 0)$ 이다.

그리고 J5_RightHip의 위치는 $t = (0.40926, 0, -1.5038)$ 이다.

$$R = [x_5, y_5, z_5]$$

$$= \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}$$

$$t = [0.40926, 0, -1.5038]^T$$

T1(초기 자세)은 아래와 같다.

0	0	1	0.40926
0	1	0	0
-1	0	0	-1.5038
0	0	0	1

f. J1_Root → J6_LeftHip (1T_6)

J6_LeftHip의 축은 $x_6 = -Z$, $y_6 = +Y$, $z_6 = +X$ 로 정의했다. 이를 J1 기준 벡터로 쓰면, $x_6 = (0, 0, -1)$, $y_6 = (0, 1, 0)$, $z_6 = (1, 0, 0)$ 이다.

그리고 J6_LeftHip의 위치는 $t = (-0.40926, 0, -1.5038)$ 이다.

$$R = [x_6, y_6, z_6]$$

$$= \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}$$

$$t = [-0.40926, 0, -1.5038]^T$$

T1(초기 자세)은 아래와 같다.

0	0	1	-0.40926
0	1	0	0
-1	0	0	-1.5038
0	0	0	1

5. 관절 회전 효과: 최종 T

위에서 계산한 T1값들(${}^1T_2 \sim {}^1T_6$)은 Initial Pose, 즉 관절이 회전되지 않은 기본 자세의 변환행렬이다. 과제 Correction 공지에서는 Initial Pose의 R과 t로 T1을 구성한 다음, joint rotation의 효과를 별도로 계산하여 최종 변환을 $T = T_2 \times T_1$ 형태로 표현하라고 설명한다. 본 보고서에서는 각 joint의 local z축을 회전축으로 배정했으므로, joint i가 θ_i 만큼

회전할 때의 회전 효과는 z축 회전행렬로 표현할 수 있다.

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

T2 =

$$\begin{bmatrix} \cos \theta_i & -\sin \theta_i & 0 & 0 \\ \sin \theta_i & \cos \theta_i & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

따라서 joint i의 최종 변환은 아래와 같이 표현할 수 있다. 여기서 T1은 앞 절에서 계산한 initial pose의 변환 행렬이다.

$$T(\theta_i) = T2 \cdot T1$$

예를 들어, 오른쪽 어깨 관절의 경우, Initial pose의 T1에 오른쪽 어깨 회전 θ_3 에 대한 $T2(\theta_3)$ 를 곱하여 회전 후의 ${}^1T_3(\theta_3)$ 를 표현한다. 실제 Unity 구현에서는 이 수식을 직접 행렬 곱으로 계산하지 않고, J3_RightShoulder Empty를 회전시키는 방식으로 동일한 관절 회전 효과를 구현했다.

- a. [4.변환 행렬 T 계산]에서 구한 T1을 이용해서 최종 변환 행렬 $T(T2 \cdot T1)$ 을 구한 결과 (a)~(f)

좌표 T 매트. ($T = T_2 \cdot T_1$)

(a) 0T_1
$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

(b) 1T_2
$$\begin{bmatrix} \sin\theta & \cos\theta & 0 & 0 \\ -\cos\theta & \sin\theta & 0 & 0 \\ 0 & 0 & 1 & (1.493) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

(c) 1T_3
$$\begin{bmatrix} \cos\theta & 0 & \sin\theta & 0.809\cos\theta \\ \sin\theta & 0 & -\cos\theta & 0.809\sin\theta \\ 0 & 1 & 0 & 0.46636 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

(d) 1T_4
$$\begin{bmatrix} -\cos\theta & 0 & \sin\theta & -0.79948\cos\theta \\ -\sin\theta & 0 & -\cos\theta & -0.79948\sin\theta \\ 0 & -1 & 0 & 0.46636 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

(e) 1T_5
$$\begin{bmatrix} 0 & -\sin\theta & \cos\theta & 0.40926\cos\theta \\ 0 & \cos\theta & \sin\theta & 0.40926\sin\theta \\ -1 & 0 & 0 & -1.5038 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

(f) 1T_6
$$\begin{bmatrix} 0 & -\sin\theta & \cos\theta & -0.40926\cos\theta \\ 0 & \cos\theta & \sin\theta & -0.40926\sin\theta \\ -1 & 0 & 0 & -1.5038 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

6. Unity import 및 Animation 구현

a. Unity import

Blender에서 제작한 캐릭터를 FBX 파일로 export한 뒤, Unity 프로젝트에 import했다.

Import 후 Unity Hierarchy에서 J1_Root 아래에 Body, J2_Neck, J3_RightShoulder,

J4_LeftShoulder, J5_RightHip, J6_LeftHip이 유지되는지 확인했다. Blender는 Z-up 좌표

계를 사용하고 Unity는 Y-up 좌표계를 사용하므로, FBX import 과정에서 캐릭터가 바닥 방

향을 향하거나 축이 다르게 보일 수 있다. 본 구현에서는 Unity의 최상위 캐릭터 root에 X축 -90도 보정을 적용하여 캐릭터가 정상적으로 서 있도록 조정했다. 또한 FBX import시 Empty(J1_Root)가 Unity Hierarchy에서 자동으로 캐릭터 루트로 병합되는 현상이 발생하여, Unity에서 J1_Root Empty Object를 직접 추가하고 기존 오브젝트들을 자식으로 재배치했다.

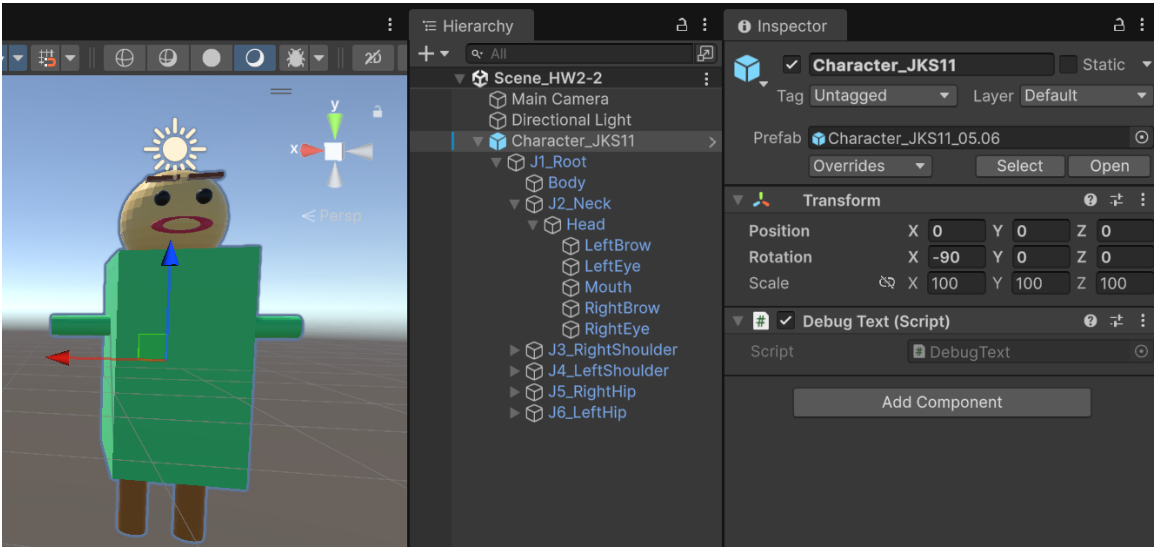
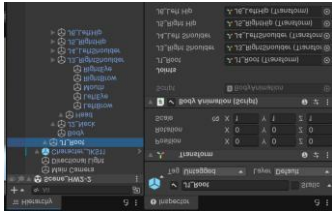
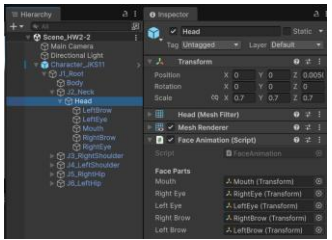
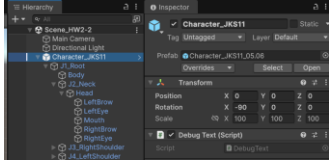


Figure 2: Unity Hierarchy 및 Inspector 화면

b. Script 구성

Script 파일	담당 기능	부착 오브젝트	제어 대상
BodyAnimation.cs	걷기, 인사, 손 흔들기	J1_Root 	J1~J6 Joint Empty
FaceAnimation.cs	기쁨, 슬픔, 화남, 말하기, 눈 깜빡임	Head(Mesh) 	Mouth, LeftEye, RightEye, LeftBrow, RightBrow Mesh

DebugText.cs	키 조작 안내 텍스트 화면 표시		OnGUI() 화면 출력
--------------	----------------------	--	------------------

c. Body Animation 구현(BodyAnimation.cs)

Body animation은 BodyAnimation.cs로 구현했다. 핵심은 팔, 다리 mesh를 직접 회전시키는 것이 아니라 J3_RightShoulder, J4_LeftShoulder, J5_RightHip, J6_LeftHip과 같은 joint Empty를 회전시키는 것이다. 이렇게 하면 joint를 중심으로 child mesh가 자연스럽게 따라 움직인다.

키	동작	구현 설명
Q	걷기(Walk)	양쪽 hip joint를 서로 반대 위상으로 회전시켜 다리가 교차로 흔들리는 동작을 만든다.
B	인사(Bow)	J1_Root를 앞으로 기울여 상체를 숙이는 동작을 만든다.
H	손 흔들기 (Wave hand)	J3_RightShoulder를 주기적으로 회전시켜 오른팔이 반복적으로 흔들리도록 한다.

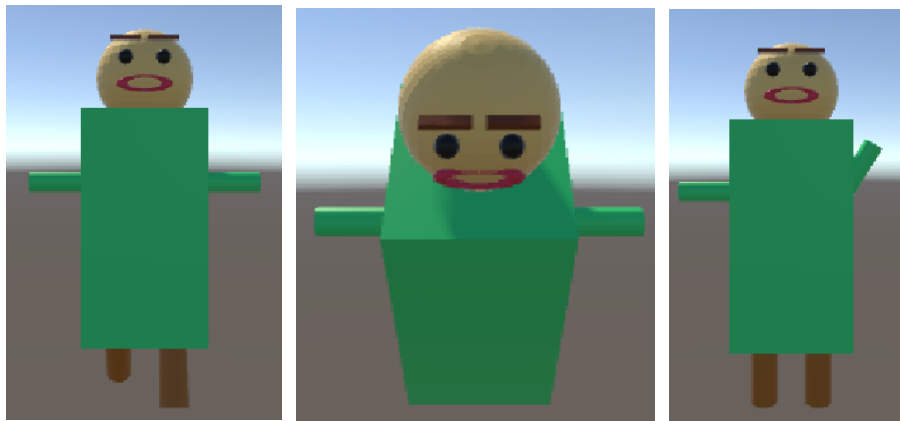


Figure 3: 왼쪽부터 순서대로 Walk/ Bow/ Wave hand 실행 화면

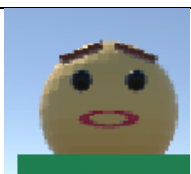
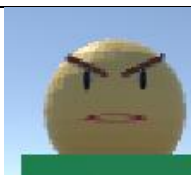
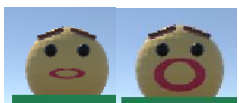
d. Facial Expression Animation 구현(FaceAnimation.cs)

Facial expression animation은 FaceAnimation.cs로 구현했다. FaceAnimation.cs는 Head Mesh에 부착하며 mouth, rightEye, leftEye, rightBrow, leftBrow 슬롯에 해당 오브젝트를

연결한다.

눈, 눈썹, 입은 Head의 child object로 구성되어 있고, 표정 변화는 각 얼굴 파츠의 local scale, local rotation, local position을 script에서 변경하는 방식으로 구현했으며, 표정 전환 시 ResetFace()로 초기화 후 새 표정을 적용한다.

입(Mouth)은 Torus로 모델링하였다. Scale 변경만으로는 자연스러운 웃는 입(U 형태) 표현이 어렵기 때문에, Joy 표정에서 Torus를 X축으로 60도 회전시켜 윗부분이 머리 Mesh 안에 파묻히게 하고 아랫부분만 보이게 하는 방식으로 스마일 형태를 구현하였다.

키	동작	구현 설명	모습
	디폴트 표정 모습		
1	기쁨, 웃음 (Laugh)	눈썹이 올라가고, 눈은 반쯤 감긴 상태에서 양끝이 내려가게 표현하며, 입은 웃는 모습을 만들어 Laugh를 표현한다.	
2	슬픔 (Sad)	눈썹 양끝은 내려가고, 입은 작아져서 Sad를 표현한다	
3	화남/썩그림 (Angry/Frown)	눈썹 양끝이 올라가고, 입은 세로로만 작아지며, 눈은 세로로 길어져서 Angry(Frown)를 표현한다.	
M	말하기 (Speaking)	입만 작아졌다 커졌다를 반복하면서 Speaking을 표현한다. Laugh/Sad/Angry 표정에서 모두 적용된다.	 1) Sad에서 적용  2) Angry에서 적용

			
R	초기화 (Reset)	디폴트 표정으로 돌아간다.	
자동	눈 깜빡임 (Blink)	눈이 빠르게 반정도로 작아졌다가 다시 원래 상태로 돌아오면서 Blink를 표현한다. 모든 표정에서 자동으로 적용된다.	

e. 키 조작 안내(DebugText.cs)

DebugText.cs는 OnGUI()를 사용하여 게임 화면 좌측에 키 조작 안내 텍스트를 실시간으로 표시한다. 내용은 아래와 같이 고정 문자열로 설정하였다.

Body animation

Q: Walk, B: Bow, H: Wave Hand

Facial expression

1: Laugh, 2: Sad, 3: Angry(Frown), M: Speaking, R: Reset

